

# Flask CRUD API with JWT Authentication

---

A simple Flask REST API implementing CRUD operations on users with JWT authentication, role-based access control, HTTPS support, and request logging.

## Features

---

- User authentication with JWT
- Role-based authorization (Admin/User)
- CRUD operations on users
- HTTPS using a self-signed certificate
- Request logging
- Protected log access

## Installation

---

Clone the project:

```
git clone <repository-url>
cd <repository>
```

Install dependencies:

```
pip install flask flask-jwt-extended
```

Run the application:

```
python crud.py
```

The API starts on:

```
https://127.0.0.1:5000
```

Since Flask uses an **adhoc SSL certificate**, your browser or API client may warn that the certificate is self-signed.

---

## Authentication

---

Authentication is performed using JSON Web Tokens (JWT).

# Login

## Endpoint

```
POST /login
```

## Request

```
{
  "nom": "Alice",
  "password": "1234"
}
```

## Response

```
{
  "token": "<JWT_TOKEN>"
}
```

Use the returned token in every protected request.

Example HTTP header:

```
Authorization: Bearer <JWT_TOKEN>
```

# Default Users

| Username | Password | Role  |
|----------|----------|-------|
| Alice    | 1234     | admin |
| Bob      | 1234     | user  |

# API Endpoints

## Get all users

```
GET /users
```

Authentication:

Not required

Example response:

```
[
  {
    "id":1,
    "nom":"Alice",
    "email":"alice@example.com"
  }
]
```

---

## Get one user

---

```
GET /users/<id>
```

Authentication:

Not required

Example:

```
GET /users/1
```

---

## Create a user

---

```
POST /users
```

Authentication:

JWT required

Authorization:

Admin only

Request:

```
{
  "nom":"John",
  "email":"john@example.com",
  "password":"1234"
}
```

Example response:

```
{
  "id": 3,
  "nom": "John",
  "email": "john@example.com",
  "password": "1234"
}
```

If a non-admin user attempts the request:

```
403 Forbidden
```

Response:

```
{
  "erreur": "Pas assez de privilèges"
}
```

---

## Update a user

```
PUT /users/<id>
```

Authentication:

Not required

Request:

```
{
  "nom": "Updated Name",
  "email": "updated@example.com"
}
```

---

## Delete a user

```
DELETE /users/<id>
```

Authentication:

Not required

Example response:

```
{
  "message": "User 3 supprimé"
}
```

```
}
```

---

## Read application log

---

```
GET /log
```

Authentication:

JWT required

Authorization:

Admin only

Returns the content of:

```
crud.log
```

---

## Logging

---

Every incoming request is logged.

Example log entry:

```
2025-03-02 14:20:13
Connexion depuis IP 127.0.0.1 à l'url https://127.0.0.1:5000/users selon la méthode GET
```

The log file is stored as:

```
crud.log
```

---

## Security Setup

---

### JWT Authentication

---

The API uses **Flask-JWT-Extended**.

Configuration:

```
app.config["JWT_SECRET_KEY"] = "ma_cle_secrete"
```

JWT tokens contain:

- Username (identity)
- User role

Example payload:

```
{
  "sub": "Alice",
  "role": "admin"
}
```

Protected routes require:

```
Authorization: Bearer <token>
```

---

## Role-Based Access Control

---

Two roles exist:

- admin
- user

Only administrators can:

- Create users
- Read application logs

Regular users receive:

```
403 Forbidden
```

---

## HTTPS

---

The application runs using:

```
ssl_context="adhoc"
```

This automatically generates a temporary self-signed SSL certificate.

All communications are encrypted.

For production, replace it with a certificate issued by a trusted Certificate Authority.

---

# Request Logging

---

Every HTTP request records:

- Client IP
- Requested URL
- HTTP method
- Timestamp

Useful for:

- Auditing
  - Debugging
  - Security monitoring
- 

## Example Workflow

---

### 1. Login

---

POST /login

Receive:

JWT Token



### 2. Send protected request

---

Authorization: Bearer eyJhbGc...



POST /users



User created

---

# Security Considerations

---

This project is intended for educational purposes.

Several improvements should be made before production deployment:

- Store passwords using secure hashing (e.g., bcrypt) instead of plain text.
- Move the JWT secret key to environment variables rather than hardcoding it.
- Protect all modification endpoints ( `PUT` and `DELETE` ) with JWT authentication and role checks.
- Validate incoming request data and handle invalid or missing fields.
- Return appropriate error responses when authentication fails or a user is not found.
- Use a database instead of in-memory storage to persist user data.
- Replace the self-signed HTTPS certificate with a trusted certificate issued by a Certificate Authority.
- Implement token expiration and refresh tokens for better session management.
- Add rate limiting to reduce the risk of brute-force attacks.

---

## Technologies

- Python
- Flask
- Flask-JWT-Extended
- HTTPS (SSL)
- JWT Authentication
- REST API
- Logging